

Chapter 12

Transaction Processing

1

Database Write Transactions per Second

 amazon.com	75,000
Booking.com	20,000
JPMorganChase 	6,000
VISA	4,000
UNITED 	2,000

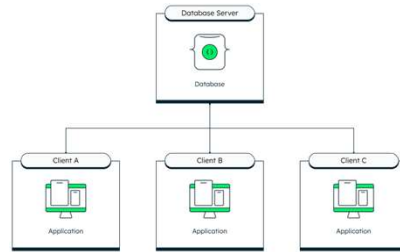
2

Transaction Processing

Transaction: a logical unit of database processing.

Relevant when many *concurrent* users are executing transactions on the same database:

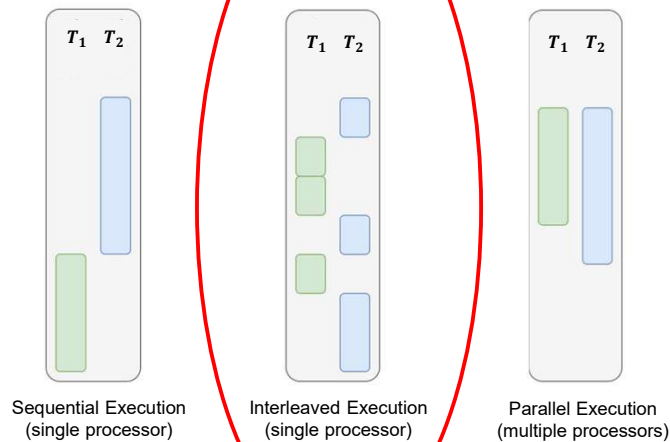
- Airline and hotel reservations
- Online shopping
- Banking
- Stock market trading
- Store checkout



3

Concurrency

Allows better utilization of system resources (CPU, RAM, DISK)



4

Transaction Operations

read_item(X)

- Reads a database item named X into a program variable named X

write_item(X)

- Writes the value of program variable X into the database item named X

Read set of a transaction = Set of all items read
Write set of a transaction = Set of all items written

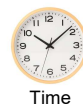
5

Transactions and Schedules

Transactions

T_1	T_2
read_item(X); $X := X - N$; write_item(X); read_item(Y); $Y := Y + N$; write_item(Y);	read_item(X); $X := X + M$; write_item(X);

Intended effect:
 $X := X + M - N$
 $Y := Y + N$



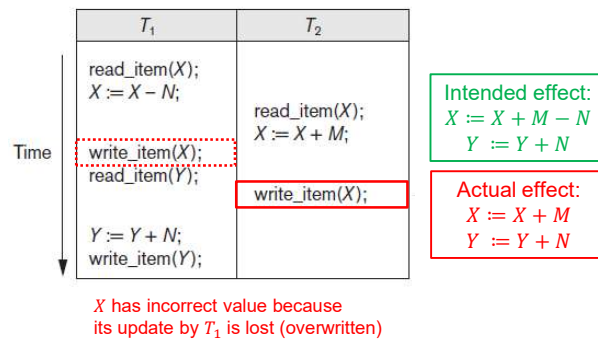
Possible Schedule

T_1	T_2
read_item(X); $X := X - N$;	
	read_item(X); $X := X + M$;
write_item(X); read_item(Y);	
	write_item(X);
$Y := Y + N$; write_item(Y);	

6

The Lost Update Problem

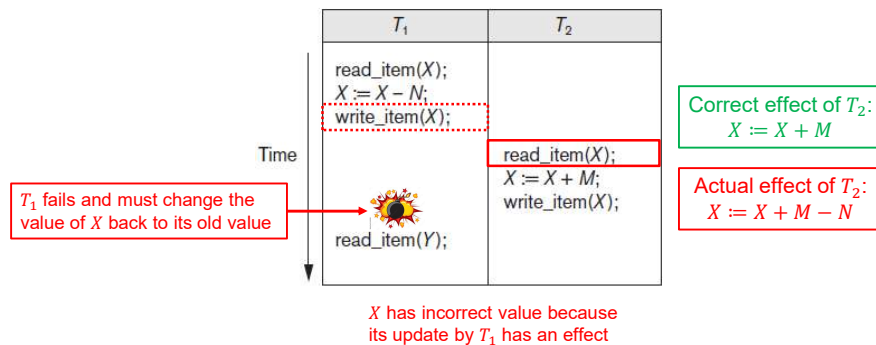
Occurs when T_1 and T_2 that access the same database items have their operations interleaved in a way that some are overwritten before they can have an effect.



7

The Temporary Update (Dirty Read) Problem

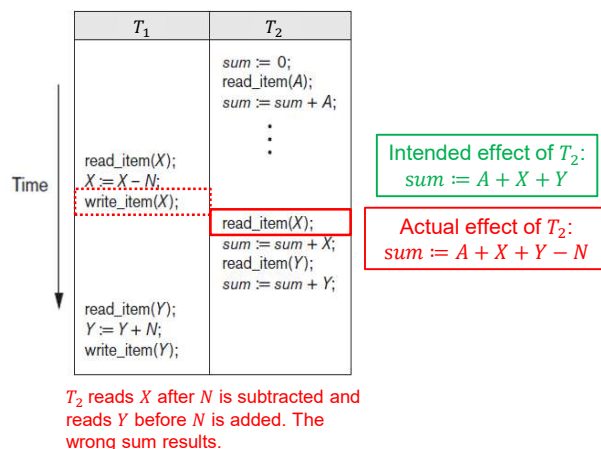
Occurs when T_1 updates a database item and then fails for some reason. The updated item is accessed by T_2 before it is changed back to its original value.



8

The Incorrect Sum Problem

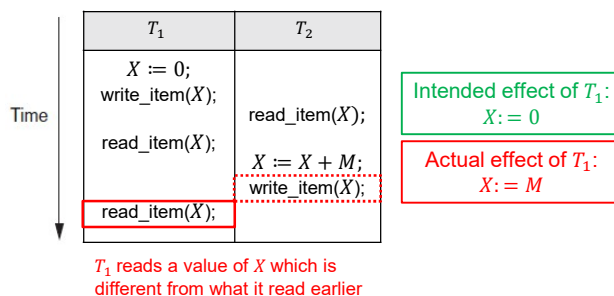
If T_1 is calculating an aggregate summary function on a number of values while T_2 is updating some of these values, T_1 may use some values before they are updated and others after they are updated.



9

The Inconsistent Read Problem

If T_1 reads the same item twice, once before its value is changed by the T_2 and once after, it receives different values for the two reads of that item.



10

ACID Properties of a DBMS

Atomicity

Consistency

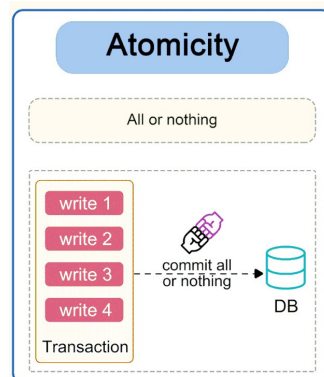
Isolation

Durability

11

Atomicity

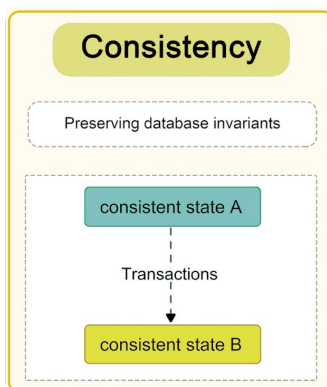
The writes in a transaction are executed as one and cannot be only partially executed. If the transaction is aborted, the writes are rolled back.



12

Consistency

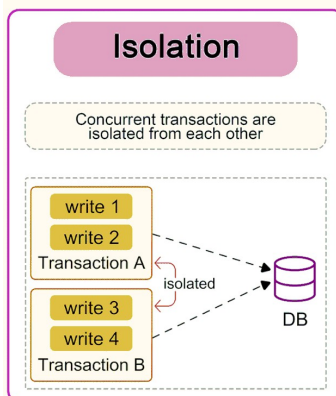
Preserve database invariants. Any data written by a transaction must be valid according to all defined rules and maintain the database in a good state.



13

Isolation

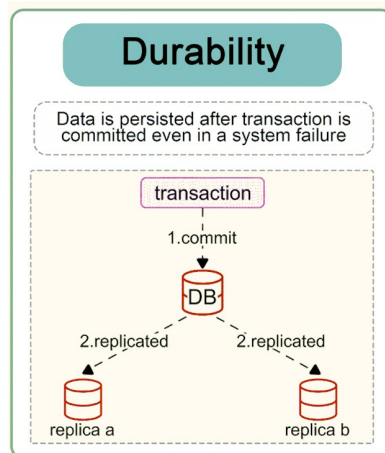
When there are concurrent writes from two different transactions, the two transactions are isolated from each other. Each transaction acts like it is the *only* transaction running on the database.



14

Durability

Data persists after a transaction is committed, even in a system failure. In a distributed system, this means the data is replicated to some other nodes.

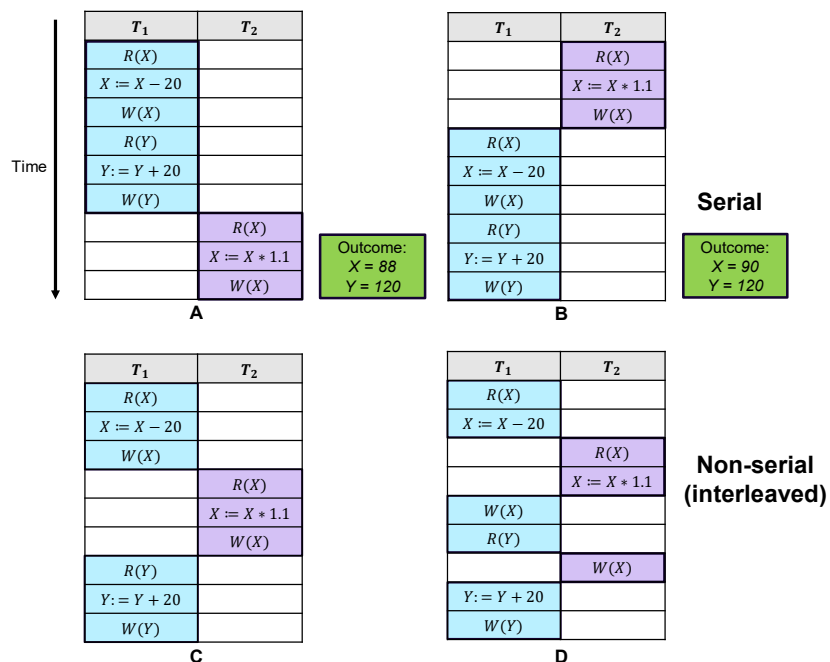


15

Transaction Schedules

T_1 : Transfer \$20 from X to Y

T_2 : Add 10% interest to X



16

Serializability

A schedule of transactions is *serializable* if it is equivalent (in its outcome) to *some* serial schedule of the same transactions.

T_1 : Transfer \$20 from X to Y

T_2 : Add 10% interest to X

Schedule C is serializable because it is equivalent to the serial Schedule A.

Serial

T_1	T_2
$R(X)$	
$X := X - 20$	
$W(X)$	
$R(Y)$	
$Y := Y + 20$	
$W(Y)$	
	$R(X)$
	$X := X * 1.1$
	$W(X)$

Start:
 $X = 100$
 $Y = 100$

Outcome:
 $X = 88$
 $Y = 120$

Serial

T_1	T_2
	$R(X)$
	$X := X * 1.1$
	$W(X)$
$R(X)$	
$X := X - 20$	
$W(X)$	
$R(Y)$	
$Y := Y + 20$	
$W(Y)$	

Outcome:
 $X = 90$
 $Y = 120$

A

B

Serializable

T_1	T_2
$R(X)$	
$X := X - 20$	
$W(X)$	
	$R(X)$
	$X := X * 1.1$
	$W(X)$
$R(Y)$	
$Y := Y + 20$	
$W(Y)$	

Outcome:
 $X = 88$
 $Y = 120$

C

Non-Serializable

T_1	T_2
$R(X)$	
$X := X - 20$	
	$R(X)$
	$X := X * 1.1$
$W(X)$	
$R(Y)$	
	$W(X)$
$Y := Y + 20$	
$W(Y)$	

Outcome:
 $X = 110$
 $Y = 120$

D

17

Conflicts

Two operations in a schedule *conflict* if

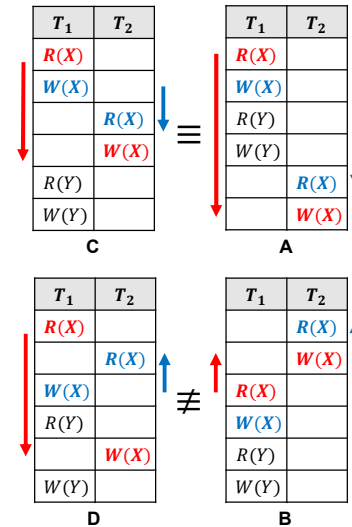
- They belong to different transactions
- They access the same item x
- At least one of the operations is a $W(x)$

RW conflicts		WW conflicts	
T_1	T_2	T_1	T_2
$R(X)$		$R(X)$	
$W(X)$		$W(X)$	
$R(Y)$			$R(X)$
$W(Y)$			$W(X)$
	$R(X)$		$R(Y)$
	$W(X)$		$W(X)$
		$R(Y)$	
		$W(Y)$	
A		B	
T_1	T_2	T_1	T_2
$R(X)$		$R(X)$	
$W(X)$			$R(X)$
	$R(X)$		$W(X)$
	$W(X)$		$R(Y)$
			$W(X)$
		$R(Y)$	
		$W(Y)$	
C		D	
T_1	T_2	T_1	T_2
$R(X)$		$R(X)$	
$W(X)$			$R(X)$
	$R(X)$		$W(X)$
	$W(X)$		$R(Y)$
			$W(X)$
		$R(Y)$	
		$W(Y)$	

18

Conflict-serializability

- Conflict equivalence
 - Two schedules are *conflict equivalent* if the relative order of any two conflicting operations is the same in both schedules
- Conflict-serializability
 - A schedule is *conflict-serializable* if it is conflict equivalent to *some* serial schedule



19

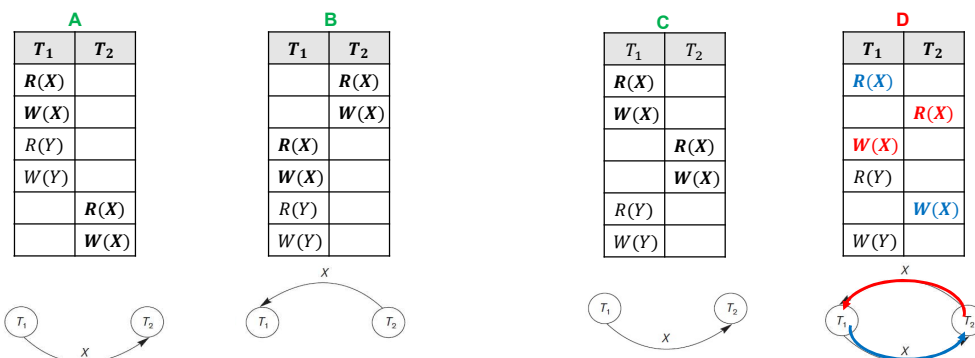
Testing for Conflict-serializability

Create a *schedule graph* G whose nodes are the transactions.

Create an edge $T_i \rightarrow T_j$ in G if:

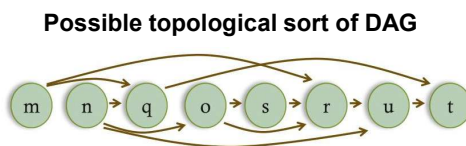
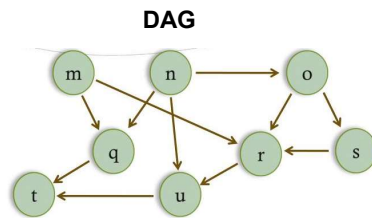
- T_j executes $R(x)$ after T_i executes $W(x)$, or
- T_j executes $W(x)$ after T_i executes $R(x)$, or
- T_j executes $W(x)$ after T_i executes $W(x)$,

A schedule is conflict-serializable iff G is a DAG - does not contain a cycle.



20

Topological Sort



A **topological sort** of a DAG is a linear ordering of its vertices such that for every directed edge $u \rightarrow v$, u comes before v in the ordering.

Kahn's Algorithm

TopoSort(G)

```

 $L \leftarrow$  Empty list // will contain the sorted elements
 $S \leftarrow$  Set of all nodes with no incoming edge
while  $S$  is not empty do
   $n := \text{pop}(S)$ 
   $\text{append}(L, n)$ 
  for each edge  $e = n \rightarrow m$  do
     $\text{remove}(G, e)$ 
    if  $m$  has no other incoming edges then
       $\text{push}(S, m)$ 

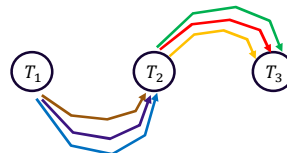
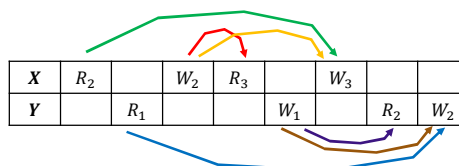
if  $G$  has edges then
  return error (graph has at least one cycle)
else
  return  $L$  (a topologically sorted order)
  
```

21

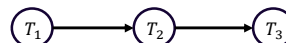
Is this schedule conflict-serializable ?

$R_2(X), R_1(Y), W_2(X), R_3(X), W_1(Y), W_3(X), R_2(Y), W_2(Y)$

T_1	T_2	T_3
	$R(X)$	
$R(Y)$		
	$W(X)$	
		$R(X)$
$W(Y)$		
		$W(X)$
	$R(Y)$	
	$W(Y)$	



YES !



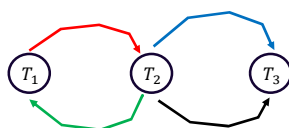
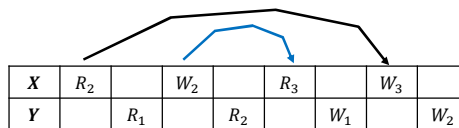
equivalent to: $R_1(Y), W_1(Y), R_2(X), W_2(X), R_2(Y), W_2(Y), R_3(Y), W_3(Y)$

22

Is this schedule conflict-serializable ?

$R_2(X), R_1(Y), W_2(X), R_2(Y), R_3(X), W_1(Y), W_3(X), W_2(Y)$

T_1	T_2	T_3
	$R(X)$	
$R(Y)$		
	$W(X)$	
	$R(Y)$	
		$R(X)$
$W(Y)$		
		$W(X)$
	$W(Y)$	



NO !

23

Concurrency Control

Methods to guarantee conflict-serializability:

- Two-Phase Locking (2PL)
- Timestamp Ordering (TO)

24

Binary Locking

A binary lock is associated with every item x in the database:

- Locked (1): x cannot be accessed – the transaction is temporarily blocked
- Unlocked (0): x can be accessed

Transactions must call **lock_item(X)** before *any* $R(X)$ and $W(X)$ and call **unlock_item(X)** immediately after *all* operations on X are complete.

```
lock_item(X):
B: if LOCK(X) = 0      (*item is unlocked*)
    then LOCK(X) ← 1  (*lock the item*)
    else
        begin
            wait (until LOCK(X) = 0
                and the lock manager wakes up the transaction);
            go to B
        end;
```

```
unlock_item(X):
    LOCK(X) ← 0;      (* unlock the item *)
    if any transactions are waiting
        then wakeup one of the waiting transactions;
```

25

transactions

T_1	T_2	T_3
$R(Y)$	$R(X)$	$R(X)$
$W(Y)$	$W(X)$	$W(X)$
	$R(Y)$	
	$W(Y)$	

schedule

T_1	T_2	T_3
	$L(X)$	
	$R(X)$	
$L(Y)$		
$R(Y)$		
$W(Y)$		
	$W(X)$	
	$UL(X)$	
$UL(Y)$		
	$L(Y)$	
	$R(Y)$	
		$L(X)$
		$R(X)$
		$W(X)$
		$UL(X)$
	$W(Y)$	
	$UL(Y)$	

transactions with locks

T_1	T_2	T_3
$L(Y)$	$L(X)$	$L(X)$
$R(Y)$	$R(X)$	$R(X)$
$W(Y)$	$W(X)$	$W(X)$
$UL(Y)$	$UL(X)$	$UL(X)$
	$L(Y)$	
	$R(Y)$	
	$W(Y)$	
	$UL(Y)$	

equivalent to:

$R_1(Y), W_1(Y)$	$R_2(X), W_2(X), R_2(Y), W_2(Y)$	$R_3(X), W_3(X)$
------------------	----------------------------------	------------------

26

transactions

T_1	T_2	T_3
$R(X)$	$R(X)$	$R(X)$
A	C	E
B	D	F

transactions with locks

T_1	T_2	T_3
$L(X)$	$L(X)$	$L(X)$
$R(X)$	$R(X)$	$R(X)$
A	C	E
B	D	F
$UL(X)$	$UL(X)$	$UL(X)$

$R_2(X), R_1(X), R_3(X)$

T_1	T_2	T_3
$R(X)$		
	$R(X)$	
		$R(X)$
A		
B		
	C	
		E
	D	
		F

prevented

T_1	T_2	T_3
	$L(X)$	
	$R(X)$	
	A	
	B	
	$UL(X)$	
$L(X)$		
$R(X)$		
C		
D		
$UL(X)$		
		$L(X)$
		$R(X)$
		E
		F
		$UL(X)$

forced

27

Ternary Locking

Binary locking too restrictive for databases

- Read operations on the same item do not conflict

But must have *exclusive* lock to write



Three locking operations

- $RL(X)$ read lock
- $WL(X)$ write lock
- $UL(X)$ unlock

28

Ternary (Shared/Exclusive or Read/Write) Locking

RL(X)

```
B: if LOCK(X) = "unlocked"
    then begin LOCK(X) ← "read-locked";
           no_of_reads(X) ← 1
        end
    else if LOCK(X) = "read-locked"
        then no_of_reads(X) ← no_of_reads(X) + 1
    else begin
        wait (until LOCK(X) = "unlocked"
              and the lock manager wakes up the transaction);
        go to B
    end;
```

Lock is either:

- unlocked
- read_locked(n)
- write_locked

UL(X)

```
if LOCK(X) = "write-locked"
    then begin LOCK(X) ← "unlocked";
           wakeup one of the waiting transactions, if any
        end
    else if LOCK(X) = "read-locked"
        then begin
            no_of_reads(X) ← no_of_reads(X) - 1;
            if no_of_reads(X) = 0
                then begin LOCK(X) = "unlocked";
                       wakeup one of the waiting transactions, if any
                end
        end
    end;
```

WL(X)

```
B: if LOCK(X) = "unlocked"
    then LOCK(X) ← "write-locked"
    else begin
        wait (until LOCK(X) = "unlocked"
              and the lock manager wakes up the transaction);
        go to B
    end;
```

29

Locking Conversion

Transaction that already holds a lock may convert the lock from one type to another:

Upgrade: issue a WL after a RL (if no other transaction holds a RL)

Downgrade: issue a RL after a WL

30

Serializability by Two-Phase Locking

Proceed in two phases:

- Expanding (growing) phase
 - New locks (on any items) can be acquired but none can be released
 - Lock upgrades must be done during this phase
- Shrinking phase
 - Existing locks can be released but none can be acquired
 - Lock downgrades must be done during this phase

31

Start: $X = 20, Y = 30$

T_1	T_2
$R(Y)$	$R(X)$
$R(X)$	$R(Y)$
$X := X + Y$	$Y := X + Y$
$W(X)$	$W(Y)$

Result of serial schedules:

- T_1, T_2 : $X = 50, Y = 80$
- T_2, T_1 : $X = 70, Y = 50$

Start: $X = 20, Y = 30$

T_1	T_2
$RL(Y)$	$RL(X)$
$R(Y)$	$R(X)$
$UL(Y)$	$UL(X)$
$WL(X)$	$WL(Y)$
$R(X)$	$R(Y)$
$X := X + Y$	$Y := X + Y$
$W(X)$	$W(Y)$
$UL(X)$	$UL(Y)$

Do not obey 2PL

Start: $X = 20, Y = 30$

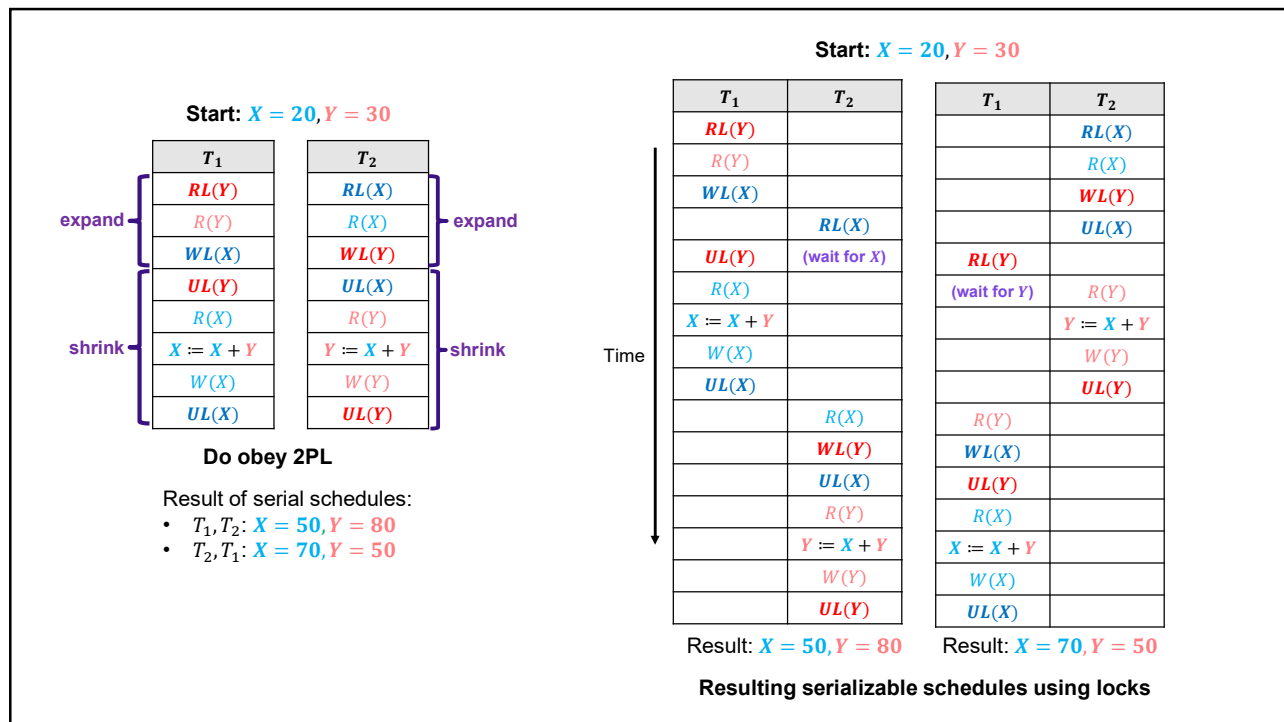
Time

T_1	T_2
$RL(Y)$	
$R(Y)$	
$UL(Y)$	
	$RL(X)$
$WL(X)$	
(wait for X)	$R(X)$
	$UL(X)$
	$WL(Y)$
	$R(Y)$
	$Y := X + Y$
	$W(Y)$
	$UL(Y)$
$R(X)$	
$X := X + Y$	
$W(X)$	
$UL(X)$	

Result: $X = 50, Y = 50$

Resulting nonserializable schedule using locks

32



33

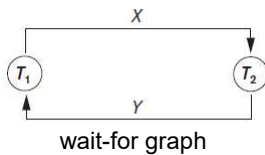
Serializability by Two-Phase Locking

- If every transaction in a schedule follows the two-phase locking protocol, the schedule is **guaranteed** to be serializable
- Two-phase locking may limit the amount of concurrency that can occur in a schedule
- Some serializable schedules will be prohibited by the two-phase locking protocol

34

Deadlock

Deadlock can occur when transaction T_1 is waiting for an item locked by transaction T_2 and vice-versa. Both transactions are stuck.



Start: $X = 20, Y = 30$

T_1	T_2
$RL(Y)$	$RL(X)$
$R(Y)$	$R(X)$
$WL(X)$	$WL(Y)$
$UL(Y)$	$UL(X)$
$R(X)$	$R(Y)$
$X := X + Y$	$Y := X + Y$
$W(X)$	$W(Y)$
$UL(X)$	$UL(Y)$

Do obey 2PL

Result of serial schedules:

- T_1, T_2 : $X = 50, Y = 80$
- T_2, T_1 : $X = 70, Y = 50$

Start: $X = 20, Y = 30$

[illegible]

Deadlock Prevention

Ideal protocols

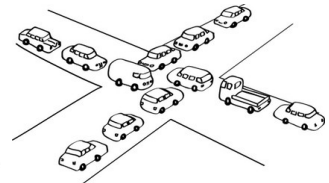
- Every transaction locks *all* items it needs in advance. If one cannot be obtained, none are locked.
- Order all items in the database. Transaction that needs several items will lock them in that order

Both approaches impractical

Timestamp based locking

- wait-die:
If lock attempted by T_2 but prevented by T_1 , abort T_2 if younger than T_1
- wound-wait:
If lock attempted by T_2 but prevented by T_1 , abort T_1 if younger than T_2

Restart after abort with same timestamp



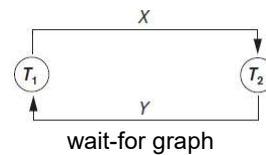
Deadlock Resolution

Deadlock detection

- Detect cycle in wait-for graph
- Abort youngest transaction in cycle

Timeout

- If transaction waits longer than a predefined time, system aborts it, irrespective of whether deadlock actually exists.



37

Timestamp Ordering (TO)

Timestamp

- Unique identifier $TS(T)$ assigned by the DBMS to transaction T
- Usually transaction **start time**

Enforce serializability of the transactions based on their timestamps

Concurrency control based on timestamps does not use locks

- Deadlocks cannot occur

38

Timestamp Ordering

Timestamp ordering

- Allows interleaving of transaction operations
- Must ensure timestamp order is followed for each pair of conflicting operations

Two dynamic timestamp-based values are maintained for each database item X :

- $RTS(X) = \max\{TS(T_i) : T_i \text{ has read } X\}$
- $WTS(X) = \max\{TS(T_i) : T_i \text{ has written } X\}$

Synchronization of T_i is achieved by comparing $RTS(X)$ and $WTS(X)$ with $TS(T_i)$ and aborting T_i as necessary.

A successful schedule (without aborts) is conflict-equivalent to a serial schedule in the order of the timestamps.

39

Timestamp Ordering

Basic TO algorithm:

- If conflicting operations detected, later operation rejected by aborting transaction that issued it and restarted with new timestamp
- Schedules produced guaranteed to be conflict-serializable
- **Starvation** may occur: A transaction is repeatedly aborted because a stream of newer, faster transactions enter the system

When transaction T issues a $W(X)$:

if $RTS(X) > TS(T)$ or $WTS(X) > TS(T)$ then

abort and roll back T

Some *younger* transaction with a timestamp greater than $TS(T)$ - and hence *after* T in the timestamp ordering - has already read or written the value of X before T had a chance to write X , thus violating the timestamp ordering.

else

execute $W(X)$

$WTS(X) := TS(T)$

When transaction T issues a $R(X)$:

if $WTS(X) > TS(T)$ then

abort and roll back T

Some *younger* transaction with timestamp greater than $TS(T)$ - and hence *after* T in the timestamp ordering - has already written the value of X before T had a chance to read X , thus violating the timestamp ordering.

else

execute $R(X)$

$RTS(X) := \max(TS(T), RTS(X))$

40

